۱-۲ سازمان سیستم کامپیوتری

یک سیستم کامپیوتری همه‌منظوره‌ی پیشرفته دارای یک یا چند پردازنده و تعدادی کنترلر دستگاه است که به‌وسیله‌ی یک **گذرگاه** مشترک که دسترسی مؤلفه‌ها و حافظه‌ی مشترک را فراهم می‌کند به هم متصل می‌باشند (شکل ۱-۲). هر یک از کنترلر دستگاه‌ها مسئول یک نوع دستگاه خاص می‌باشد (برای مثال، دیسک‌گردان، دستگاه صوتی و نمایشگر گرافیکی). بسته به کنترلر، بیشتر از یک دستگاه ممکن است متصل شود. برای نمونه، یک درگاه سیستم USB می‌تواند به یک هاب USB متصل شود که چندین دستگاه می‌تواند به آن متصل شود. کنترلر دستگاه، مقداری حافظه‌ی بافر محلی و یک مجموعه ثبات خاص‌منظوره نگه می‌دارد. کنترلر دستگاه مسئول جابجایی داده‌ها بین دستگاه‌های جانبی که کنترل می‌کند و حافظه‌ی بافر محلی می‌باشد.

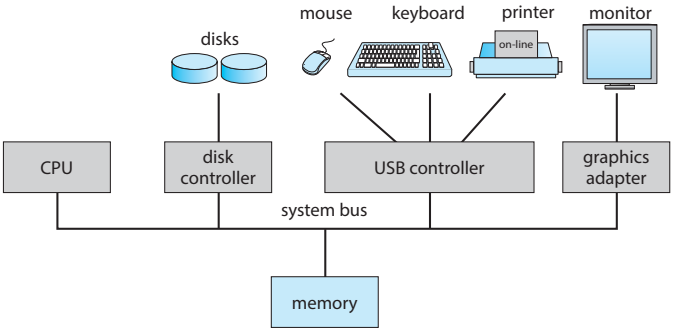
سیستم‌عامل‌ها نوعاً یک **درایور دستگاه** برای هر کنترلر دستگاه دارند. این درایور دستگاه، کنترلر دستگاه را می‌فهمد و برای بقیه‌ی سیستم‌عامل، یک رابط یک‌شکلی به آن دستگاه فراهم می‌کند. CPU و کنترلر دستگاها می‌توانند به صورت موازی در رقابت برای دوره‌های حافظه اجرا شوند. برای تضمین دسترسی مرتب به حافظه‌ی مشترک، یک کنترلر حافظه وجود دارد که دسترسی به حافظه را همگام می‌سازد.

در زیربخش‌های بعدی، برخی مبانی چگونگی عملیات یک چنین سیستمی را با تمرکز بر سه وجه کلیدی یک سیستم بیان می‌کنیم. ما با وقفه‌ها شروع می‌کنیم که به CPU هشدار می‌دهند تا توجه خود را به رویدادهایی معطوف سازد. سپس، ساختار ذخیره‌سازی و ساختار I/O را بحث می‌کنیم.

۱-۲-۱ وقفه

یک عملیات نوعی کامپیوتر را در نظر بگیرید: برنامه‌ای که I/O انجام می‌دهد. برای شروع یک عملیات I/O، درایور دستگاه، ثبات‌های مناسبی را در کنترل دستگاه بارگذاری می‌کند. کنترلر دستگاه، به نوبت خود، محتوای این ثبات‌ها را بررسی می‌کند تا تعیین کند که کدام عمل (همچون «یک کاراکتر از صفحه‌کلید بخوان») انجام شود. کنترلر، انتقال داده‌ها را از دستگاه به بافر محلی خودش شروع می‌کند. همین که انتقال داده‌ها تکمیل شد، کنترلر دستگاه به درایور دستگاه اطلاع می‌دهد که عملیات خودش را به‌اتمام رسانده است. سپس، درایور دستگاه کنترل را به سایر بخش‌های سیستم‌عامل می‌دهد، احتمالاً با برگرداندن داده‌ها یا اشاره‌گری به داده‌ها، در صورتی که عملیات خواندن بوده باشد. برای عملیات دیگر، درایور دستگاه اطلاعات وضعیتی بر می‌گرداند همچون «نوشتن به‌صورت موفق کامل شد» یا «دستگاه مشغول است». اما کنترلر چگونه به درایور دستگاه اطلاع می‌دهد که عملیات خودش را به‌پایان رسانده است؟ این کار از طریق یک **وقفه**[1] انجام می‌شود.

1. Interrupt



شکل ۱-۲ سیستم کامپیوتر مدرن.

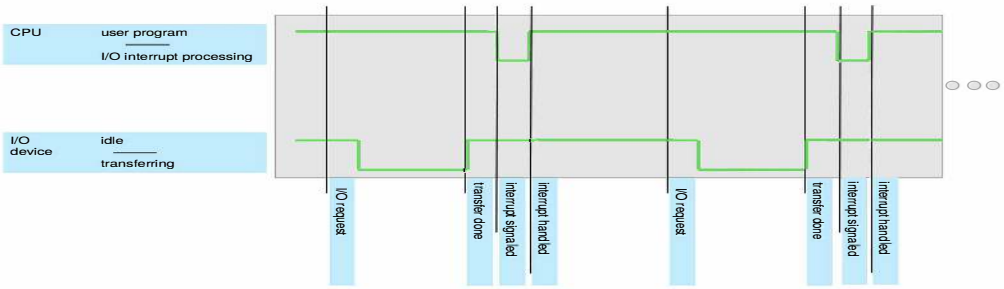
۱-۲-۱-۱ کلیات

سخت‌افزار در هر لحظه ممکن است از طریق گذرگاه سیستم، با ارسال یک سیگنال به پردازنده، یک وقفه صادر کند. (گذرگاه‌های زیادی ممکن است در یک سیستم کامپیوتری وجود داشته باشند، اما گذرگاه سیستمی است که مسیر ارتباطات مهم میان مؤلفه‌های اصلی است.) وقفه‌ها برای اهداف زیاد دیگری نیز استفاده شده و یک بخش کلیدی چگونگی تعامل سیستم‌عامل‌ها و سخت‌افزار به‌حساب می‌آیند.

زمانی که پردازنده وقفه دریافت می‌کند، کار خود را متوقف کرده و بلافاصله اجرا را به یک مکان ثابت انتقال می‌دهد. مکان ثابت، معمولاً آدرس شروع استقرار روال سرویس وقفه را در بر دارد. روال سرویس وقفه اجرا می‌شود؛ به محض تکمیل، پردازنده محاسبات وقفه دیده را از سر می‌گیرد. خط زمانی این عملیات در شکل ۱-۳ نشان داده می‌شود.

وقفه‌ها یک بخش مهم هر معماری کامپیوتر به حساب می‌آیند. هر طراحی کامپیوتر، راهکار وقفه‌ی خاص خود را دارد، اما چندین کارکرد مشترک هم در میان آنها وجود دارند. وقفه بایستی کنترل را به روال سرویس مناسب وقفه انتقال بدهد. روش سر راست انجام این انتقال، فراخوانی یک روال عمومی است تا اطلاعات وقفه را بررسی کند. این روال، به نوبت خود، روال راه‌انداز مخصوص وقفه را فراخوانی خواهد کرد. با این همه، وقفه‌ها بایستی به سرعت راه‌اندازی شوند. از آن جایی که تنها تعداد از پیش تعریف شده‌ای از وقفه‌ها امکان‌پذیرند، به منظور تسریع این عملیات، می‌توان به جای مکان ثابت، جدولی از اشاره‌گرهای روال‌های وقفه‌ها را ذخیره کرد. روال وقفه به صورت غیرمستقیم از طریق جدول، بدون نیاز به یک روال میانی فراخوانی می‌شود. جدول اشاره‌گرها معمولاً در حافظه‌ی پایین ذخیره می‌شود (صد مکان اول یا مکان‌هایی از این دست). این مکان‌ها آدرس روال‌های سرویس وقفه‌ها را برای دستگاه‌های مختلف نگه می‌دارند. سپس این آرایه‌ی، یا **بردار وقفه**[1]، آدرس‌ها مطابق شماره‌ی منحصر دستگاه‌ها ایندکس می‌شود تا با درخواست وقفه‌ای مشخص، آدرس روال سرویس وقفه برای دستگاه وقفه‌دهنده را فراهم نماید. وقفه‌های سیستم‌عامل‌های ویندوز و یونیکس، جدا از تفاوت‌هایی هم که دارند با این شیوه راه‌اندازی می‌شوند.

1. Interrupt vector



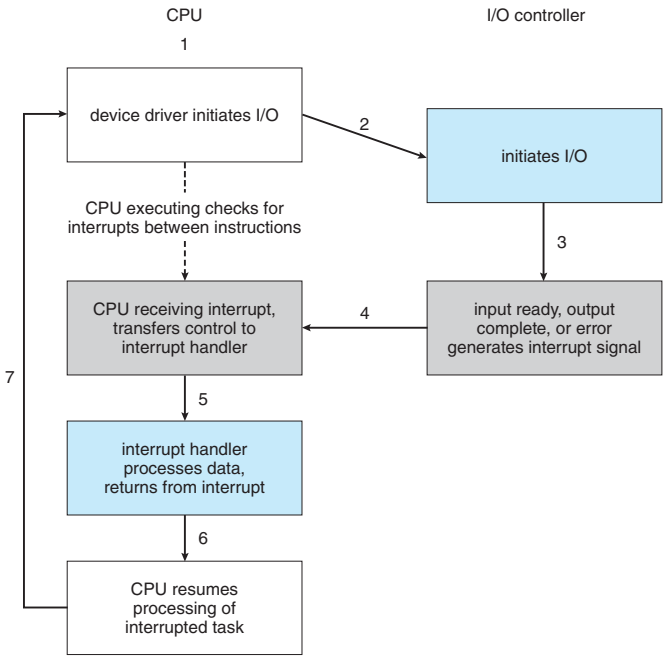
شکل ۱-۳ خط زمانی وقفه برای تک‌پردازه‌ای که خروجی دارد.

معماری وقفه بایستی آدرس دستورالعمل وقفه‌دیده را ذخیره نماید. بسیاری از طراحی‌های قدیمی، آدرس وقفه را به سادگی در یک مکان ثابت یا در مکانی که روی شماره‌ی دستگاه ایندکس می‌شد، ذخیره می‌کردند. بیشتر معماری‌های اخیر، آدرس برگشت را روی پشته‌ی سیستم ذخیره می‌کنند. اگر روال وقفه لازم دارد که حالت پردازنده را تغییر بدهد – برای نمونه، با تغییر مقادیر ثبات‌ها—بایستی مشخصاً حالت فعلی را ذخیره نماید و در ادامه، با بازگشت، حالت قبلی را بازیابی کند. بعد از سرویس‌دهی وقفه، آدرس بازگشت ذخیره‌شده، در شمارنده‌ی برنامه بارگذاری و محاسبات متوقف شده از سر گرفته می‌شوند، چنان که اصلاً وقفه‌ای صورت نگرفته است.

۱-۲-۱-۲ پیاده‌سازی

راهکار پایه‌ای وقفه به این صورت کار می‌کند. سخت‌افزار پردازنده خطی به‌نام **خط درخواست وقفه**[1] دارد که پردازنده بعد از هر بار اجرای دستورالعمل حس می‌کند. موقعی که پردازنده تشخیص می‌دهد که یک کنترلر، سیگنالی را بر روی خط درخواست وقفه گذاشته است، شماره وقفه را خوانده و با استفاده از آن شماره‌ی وقفه به‌عنوان ایندکسی برای بردار وقفه به **روال راه‌انداز وقفه**[2] می‌پرد. سپس، اجرا را در آدرس مربوط به آن ایندکس شروع می‌کند. راه‌انداز وقفه، هر حالتی را که در طی اجرای عملیاتش تغییر خواهد داد ذخیره کرده، علت وقفه را تعیین کرده، پردازش لازم را انجام داده، یک بازیابی حالت انجام داده، و یک دستورالعمل return_from_interrupt را برای برگشت پردازنده به حالت اجرایی ماقبل وقفه اجرا می‌کند. ما می‌گوییم که کنترلر دستگاه با گذاشتن یک سیگنال بر روی خط درخواست وقفه، یک وقفه *راه می‌اندازد*[3]، پردازنده وقفه را *قاپیده*[4] و به راه‌انداز وقفه *اعزام می‌کند*[5]، و راه‌انداز با سرویس‌دهی به دستگاه وقفه را *پاک می‌کند*. شکل ۱-۴، دور I/O وقفه‌رانده را خلاصه می‌نماید.

1. Interrupt-request line
2. Interrupt-handler routine
3. Raises
4. Catches
5. Dispatches



شکل ۱-۴ دور I/O وقفه‌رانده

راهکار پایه‌ای وقفه که بیان شد، پردازنده را قادر می‌سازد تا به یک رویداد ناهمگام پاسخ دهد، زمانی که کنترلر دستگاهی برای سرویس آماده می‌شود. با این وجود، در یک سیستم‌عامل مدرن، ما به امکانات راه‌اندازی وقفه‌ی پیچیده‌ای نیاز داریم.

۱- نیاز به این توانایی داریم تا راه‌اندازی وقفه را در طی پردازش بحرانی به‌تأخیر بیندازیم.

۲- به یک روش کارآمد نیاز داریم تا به یک راه‌انداز وقفه‌ی مناسب برای یک دستگاه اعزام داشته باشیم.

۳- ما وقفه‌های چندسطحی را نیاز داریم تا این که سیستم‌عامل بتواند وقفه‌های میان الویت بالا و پایین را تمییز داده و بتواند با درجه‌ی فوریت مناسبی پاسخ‌گو باشد.

این سه امکان در سخت‌افزار کامپیوتری مدرن توسط پردازنده و **سخت‌افزار کنترلر وقفه** فراهم می‌شوند.

اکثر پردازنده‌ها دارای دو خط درخواست وقفه هستند. یکی **وقفه‌ی غیرقابل منع**[1] است که برای رویدادهایی همچون خطاهای حافظه‌ی غیرقابل احیا محفوظ است. خط دوم وقفه **قابل منع**[2] است: این خط می‌تواند توسط پردازنده قبل از اجرای توالی دستورالعمل‌های بحرانی که بایستی وقفه نداشته باشند خاموش شود. وقفه‌ی قابل منع توسط کنترلر دستگاه‌ها برای درخواست سرویس استفاده می‌شود.

1. Nonmaskable interrupt
2. Maskable

به یاد بیاورید که هدف از یک راهکار وقفه‌ی برداری، کاهش نیاز به یک راه‌انداز وقفه‌ی تکی برای جستجوی تمامی منابع ممکن وقفه‌ها به‌منظور تعیین این که کدام نیاز به سرویس دارد می‌باشد. با این وجود، کامپیوترها در عمل بیشتر از آن تعداد عناصری که (و، از این رو، راه‌انداز وقفه‌هایی) در بردار وقفه آدرس‌دهی می‌کنند دارند. شیوه‌ی مرسوم حل این مسئله استفاده از **زنجیرسازی وقفه**[1] است که در آن، هر عنصر بردار وقفه به سر لیستی از راه‌اندازهای وقفه اشاره می‌کند. موقعی که یک وقفه‌ای سر می‌زند، راه‌اندازهای روی لیست متناظر یک‌به‌یک صدا زده می‌شوند تا این که یکی پیدا شود که بتواند درخواست را سرویس دهد. این ساختار، مصالحه‌ای میان سربار یک جدول وقفه‌ی عظیم و ناکارآمدی اعزام به یک راه‌انداز وقفه‌ی تکی است.

شکل ۱–۵، طراحی بردار وقفه را برای پردازنده‌های اینتل نمایش می‌دهد. رویدادهای ۰ تا ۳۱ که غیرقابل منع هستند برای علامت‌دادن شرایط مختلف خطا استفاده می‌شوند. رویدادهای ۳۲ تا ۲۵۵ که قابل منع هستند برای اهدافی همچون وقفه‌های تولیدی دستگاه استفاده می‌شوند.

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

شکل ۱–۵ جدول بردار رویداد پردازنده‌ی اینتل

راهکار وقفه یک سیستمی از **سطوح الویت وقفه** نیز پیاده‌سازی می‌کند. این سطوح، پردازنده را قادر می‌سازند تا راه‌اندازی وقفه‌های الویت پایین را بدون منع تمامی وقفه‌ها به‌تأخیر انداخته و برای یک وقفه‌ی الویت بالا، پس‌گیری اجرای یک وقفه‌ی الویت پایین را فراهم کنند.

به‌طور خلاصه، وقفه‌ها در سرتاسر سیستم‌عامل‌های مدرن برای رسیدگی به رویدادهای ناهمگام (و برای سایر اهدافی که در کل کتاب بحث خواهیم کرد) استفاده می‌شوند. کنترلر –

1. Interrupt chaining

دستگاه‌ها و خطاهای سخت‌افزاری وقفه‌ها را به‌راه می‌اندازند. کامپیوترهای مدرن برای این که قادر باشند کار با بیشترین فوریت را اول انجام دهند از سیستمی از الویت‌های وقفه استفاده می‌کنند. چون وقفه‌ها برای پردازش با حساسیت زمانی این اندازه سنگین از وقفه‌ها استفاده می‌کنند، راه‌اندازی کارآمد وقفه برای کارایی خوب سیستم ضرورت دارد.

۱-۲-۲ ساختار ذخیره‌سازی

پردازنده می‌تواند دستورالعمل‌ها را فقط از حافظه بردارد، لذا هر یک از برنامه‌ها بایستی برای اجرا در آنجا ذخیره شوند. کامپیوترهای همه‌منظوره اکثر برنامه‌های خود را از حافظه‌ی با قابلیت نوشتن مجدد به نام حافظه‌ی اصلی (که **حافظه‌ی با قابلیت دسترسی تصادفی** یا **RAM** نیز نامیده می‌شود) اجرا می‌کنند. حافظه‌ی اصلی عمدتاً با یک فناوری نیمه‌هادی به نام **حافظه‌ی با دسترسی تصادفی پویا** (**DRAM**) پیاده‌سازی می‌شود.

کامپیوترها از اشکال دیگر حافظه هم استفاده می‌کنند. برای مثال، اولین برنامه‌ای که با روشن‌کردن کامپیوتر اجرا می‌شود **برنامه‌ی بالاآورنده**[1] است که در ادامه‌ی کار، سیستم‌عامل را بارگذاری می‌کند. از آنجایی که RAM **میراست**[2]—محتویات خود را در زمانی که کامپیوتر خاموش شده یا به هر دلیلی، برق آن قطع می‌شود از دست می‌دهد—نمی‌توانیم به آن اعتماد کنیم که برنامه‌ی بالاآورنده را حفظ می‌کند. در عوض، کامپیوتر برای این منظور و برخی اهداف دیگر از حافظه‌ی فقط‌خواندنی برنامه‌پذیر پاک‌شدنی الکتریکی[3] (EEPROM) و سایر اَشکال سفت‌افزار[4]—حافظه‌ای که به‌ندرت نوشته می‌شود و نامیراست—استفاده می‌کند. EEPROM قابل تغییر است اما نمی‌تواند به صورت مکرر تغییر یابد. به‌علاوه، سرعت آن پایین است و لذا بیشتر شامل برنامه‌ها و داده‌های ایستا می‌باشد که زیاد استفاده نمی‌شوند. برای مثال، آیفون[5] برای ذخیره‌کردن شماره‌ی سریال‌ها و اطلاعات سخت‌افزاری مربوط به دستگاه از EEPROM استفاده می‌کند.

تعاریف و نمادهای ذخیره‌سازی

واحد اساسی ذخیره‌سازی کامپیوتر بیت است. یک بیت می‌تواند یکی از دو مقدار، ۰ و ۱ را داشته باشد. تمامی حافظه‌های دیگر کامپیوتر بر اساس مجموعه‌هایی از بیت‌هاست. تعجب‌آور است که اگر کامپیوتری بیت کافی داشته باشد چه چیزهای زیادی را می‌تواند نمایش بدهد: اعداد، حروف، تصاویر، فیلم‌ها، صداها، اسناد و برنامه‌ها و از این دست. یک بایت، ۸ بیت است و در اکثر کامپیوترها به عنوان کوچک‌ترین قالب ذخیره‌سازی مناسب است. برای مثال، اکثر کامپیوترها دستورالعملی برای انتقال یک بیت ندارند، اما برای انتقال بایت دارند. اصطلاح کلمه که کمتر متداول است، واحد داده‌ای بومی یک معماری کامپیوتر مشخص است. یک کلمه عموماً از یک یا چند بایت شکل می‌گیرد. برای مثال،

1. Bootstrap program
2. Volatile
3. Electrically erasable programmable read-only memory (EEPROM)
4. Firmware
5. iPhone

کامپیوتری که ثبات‌های ۶۴ بیتی و آدرس‌دهی حافظه‌ی ۶۴ بیتی دارد، نوعاً دارای کلمات ۶۴ بیتی (۸ بایتی) است. یک کامپیوتر، عملیات فراوانی را به جای بایت با اندازه‌ی کلمه‌ی بومی خود در هر واحد زمان اجرا می‌کند.

ذخیره‌سازی کامپیوتر، در کنار بازده کامپیوتر، عموماً با بایت و مجموعه‌ای از بایت‌ها سنجیده و دستکاری می‌شود. یک کیلوبایت، یا KB، 1024 بایت؛ یک مگابایت، یا MB، 1024^2 بایت؛ یک گیگابایت، یا GB، 1024^3 بایت؛ یک ترابایت یا TB، 1024^4 بایت و یک پتابایت، یا PB، 1024^5 بایت است. سازندگان کامپیوتر، اغلب این اعداد را گرد کرده و به هر مگابایت، ۱ میلیون بایت و به هر گیگابایت، ۱ بیلیون بایت می‌گویند. سنجش‌های شبکه در مورد این قاعده‌ی کلی مستثنی هستند؛ آنها به بیت مشخص می‌شوند (چون شبکه‌ها، در هر لحظه، داده‌ها را بیت به بیت جابه‌جا می‌کنند).

تمامی اشکال حافظه آرایه‌ای از بایت‌ها را فراهم می‌کنند. هر بایت، آدرس خاص خود را دارد. تعامل، از طریق دنباله‌ای از دستورالعمل‌های load یا store به آدرس‌های مشخصی از حافظه صورت می‌گیرد. در حالی‌که دستورالعمل load، یک بایت یا کلمه را از حافظه‌ی اصلی به یک ثبات داخل پردازنده انتقال می‌دهد، دستورالعمل store، محتویات یک ثبات را به حافظه‌ی اصلی منتقل می‌کند. جدا از دستورالعمل‌های مشخص load و store، پردازنده دستورالعمل‌هایی را به منظور اجرا به طور خودکار از حافظه‌ی اصلی، از محلی که در شمارنده‌ی برنامه ذخیره شده است، بارگذاری می‌کند.

دور اجرای دستورالعمل نوعی که روی یک سیستم با معماری Von Neumann اجرا می‌شود، ابتدا دستورالعملی را از حافظه واکشی کرده و آن دستورالعمل را در **ثبات دستورالعمل** ذخیره می‌کند. سپس، دستورالعمل رمزگشایی‌شده[1] و ممکن است باعث واکشی عملوندهایی نیز از حافظه شده و در برخی ثبات‌های داخلی ذخیره شوند. بعد از اجرای دستورالعمل بر روی عملوندها، ممکن است نتیجه‌ی دستورالعمل دوباره در حافظه ذخیره شود. توجه داشته باشید که واحد حافظه، تنها جریانی از آدرس‌های حافظه را می‌بیند. آن نمی‌داند که آنها چگونه (با شمارنده‌ی دستورالعمل، ایندکس، غیرمستقیم، آدرس‌های ثابت و از این دست) و به چه منظوری (واکشی داده‌ها یا دستورالعمل‌ها) تولید می‌شوند. بر این اساس، می‌توانیم از چگونگی تولید یک آدرس حافظه توسط برنامه چشم‌پوشی کنیم. تنها به دنباله‌ی آدرس‌های حافظه که توسط برنامه‌ی در حال اجرا تولید می‌شود، علاقه‌مندیم.

در حالت آرمانی می‌خواهیم که برنامه‌ها و داده‌ها به طور دایمی در حافظه‌ی اصلی ماندگار شوند. این امر در اکثر سیستم‌ها به دو دلیل زیر ناممکن است:

۱- حافظه‌ی اصلی معمولاً آن قدر کوچک است که نمی‌تواند تمامی برنامه‌ها و داده‌های مورد نیاز را برای همیشه ذخیره کند.

۲- حافظه‌ی اصلی، چنان‌چه ذکر شد، میراست—محتویات خود را با قطع برق یا به دلایل مشابه از دست می‌دهد.

1. Decode

بنابراین، اکثر سیستم‌های کامپیوتری برای گسترش حافظه‌ی اصلی، **حافظه‌ی ثانویه**[1] را فراهم می‌کنند. حافظه‌ی ثانویه قادر است مقادیر بزرگی از داده‌ها را به صورت دایمی نگه‌داری کند.

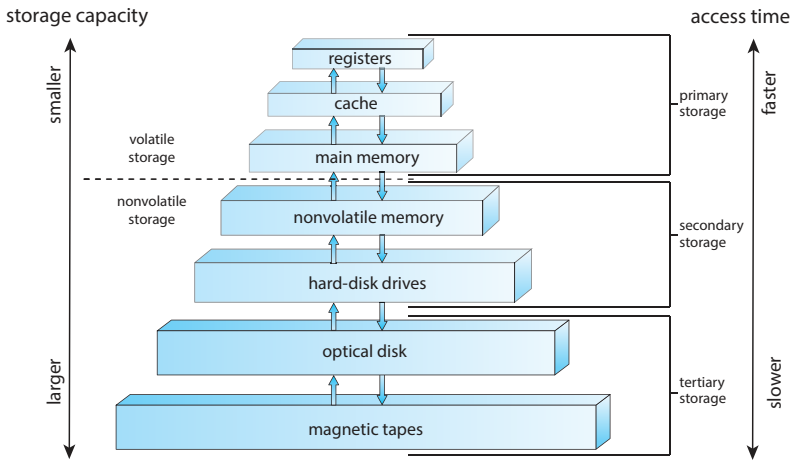
رایج‌ترین دستگاه حافظه‌ی ثانویه **دیسک‌گردان دیسک سخت**[2] (**HDD**) و **دستگاه‌های حافظه‌ی نامیرا**[3] (**NVM**) هستند که امکان ذخیره‌سازی توأم برنامه‌ها و داده‌ها را فراهم می‌کند. اکثر برنامه‌ها (سیستمی و کاربردی) در حافظه‌ی ثانویه ذخیره‌هستند تا این که داخل حافظه بار شوند. به این ترتیب، بسیاری از برنامه‌ها، حافظه‌ی ثانویه را همزمان به عنوان مبدأ و مقصد پردازش خود استفاده می‌کنند. از این‌رو، مدیریت درست حافظه‌ی ثانویه در یک سیستم کامپیوتری از اهمیت خاصی برخوردار است که در فصل ۱۱ مورد بحث قرار می‌دهیم.

با این وجود، در یک مقیاس وسیع، ساختار ذخیره‌سازی که بررسی کردیم—شامل ثبات‌ها، حافظه‌ی اصلی و حافظه‌ی ثانویه—تنها یکی از طرح‌های سیستم ذخیره‌سازی متعدد ممکن است. مؤلفه‌های ممکن دیگر عبارتند از حافظه‌ی نهانی، CD-ROM یا blu-ray، نوارهای مغناطیسی و غیره. سیستم‌هایی که کند بوده و به‌قدر کافی بزرگ هستند تنها برای اهداف خاصی—به‌عنوان مثال، برای ذخیره‌کردن کپی پشتیبان مطالب ذخیره شده بر روی سایر دستگاه‌ها—استفاده می‌شوند. هر سیستم ذخیره‌سازی، کارکردهای اصلی یعنی قابلیت ذخیره و نگه‌داری تا بازیابی داده را در آینده فراهم می‌کند. تفاوت‌های اصلی انواع سیستم‌های حافظه، به سرعت، اندازه و میرایی آنها بر می‌گردد.

انواع گسترده‌ی سیستم‌های حافظه را می‌توان بر اساس ظرفیت ذخیره‌سازی و زمان دسترسی در یک سلسله‌مراتب (شکل ۱-۶) سازماندهی کرد. به‌عنوان یک قاعده‌ی کلی، یک مصالحه‌ای میان اندازه و سرعت وجود دارد، حافظه‌های کوچک‌تر و سریع‌تر به CPU نزدیک‌ترند. چنان‌چه در شکل نشان داده می‌شود، علاوه بر تفاوت در سرعت و ظرفیت، سیستم‌های مختلف ذخیره‌سازی، میرا و یا نامیرا هستند. حافظه‌ی میرا، آن‌گونه که پیشتر عنوان شد، محتویات خود را در زمانی که برق از دستگاه قطع می‌شود از دست می‌دهد، لذا داده‌ها می‌بایست برای نگه‌داری امن در یک حافظه‌ی نامیرا نوشته شوند.

چهار سطح بالایی حافظه در شکل با استفاده از **حافظه‌ی نیمه‌هادی** ساخته می‌شوند که شامل مدارات الکترونیکی مبتنی بر نیمه‌هادی می‌باشند. دستگاه‌های NVM در سطح چهارم، دارای گونه‌های متعددی هستند اما عموماً سریع‌تر از دیسک‌های سخت می‌باشند. رایج‌ترین شکل دستگاه NVM حافظه‌ی فلش است که در دسگاه‌های موبایل همچون گوشی‌های هوشمند و تبلت استفاده می‌شوند. به‌صورت روزافزون، حافظه‌ی فلش برای ذخیره‌سازی بلندمدت در لپ‌تاپ‌ها، کامپیوترهای رومیزی، همین‌طور کارگزارها استفاده می‌شوند.

1. Secondary storage
2. Hard-disk drive (HDD)
3. Nonvolatile memory



شکل ۱-۶ سلسله‌مراتب دستگاه حافظه.

از آنجایی که حافظه نقش مهمی در ساختار سیستم‌عامل ایفا می‌کند ما به‌کرات در این کتاب به آن رجوع خواهیم کرد. در حالت کلی، ما از اصطلاحات زیر استفاده خواهیم کرد:

- ذخیره‌ی میرا به‌صورت ساده تحت عنوان **حافظه** یاد می‌شود. اگر نیاز باشد که روی یک نوع خاصی از دستگاه ذخیره‌سازی تأکید کنیم (برای مثال، ثبات)، این موضوع را صراحتاً بیان می‌کنیم.
- ذخیره‌ی نامیرا[1] محتویات خود را موقع قطع برق حفظ می‌کند که به آن NVS گفته می‌شود. بیشتر زمان‌هایی که ما روی NVS بحث می‌کنیم، منظور حافظه‌ی ثانویه است. این نوع حافظه می‌تواند در دو نوع متمایز دسته‌بندی شود:
  - **مکانیکی**: مثال‌هایی از این قبیل سیستم‌های حافظه، HDDها، دیسک‌های نوری، حافظه‌ی هولوگرافیک، و نوار مغناطیسی هستند. اگر نوع خاصی از دستگاه حافظه‌ی مکانیکی مد نظر ما باشد (برای مثال، نوار مغناطیسی)، صراحتاً اعلام خواهیم کرد.
  - **الکتریکی**: مثال‌هایی از این قبیل سیستم‌های حافظه عبارتند از حافظه‌ی فلش، FRAM، NRAM، و SSD. حافظه‌ی الکتریکی، تحت عنوان **NVM** یاد می‌شود. اگر نیاز باشد که روی نوع خاصی از یک دستگاه حافظه‌ی الکتریکی (برای مثال، SSD) تأکید کنیم، صراحتاً اعلام خواهیم کرد.

حافظه‌ی مکانیکی عموماً بزرگ‌تر و کم‌هزینه‌تر در ازای بایت از حافظه‌ی الکتریکی است. برعکس، حافظه‌ی الکتریکی نوعاً هزینه‌برتر، کوچک‌تر، و سریع‌تر از حافظه‌ی مکانیکی است.

طراحی یک سیستم حافظه‌ی کامل بایستی میان تمامی عوامل بحث شده تعادل برقرار نماید: بایستی حتی‌الامکان از حافظه‌ی گران استفاده نکند و تا حد ممکن، حافظه‌ای ارزان و پایدار فراهم

1. Nonvolatile storage